

Verification Plan

8-bit Arithmetic Logic Unit (ALU)

ALU RTL (ALU.sv) verified against UVM-style SystemVerilog testbench
Prepared for: ADI Digital Design Verification

Prepared by: Youssef Bassem Lamey

ALU Block Diagram and Specification

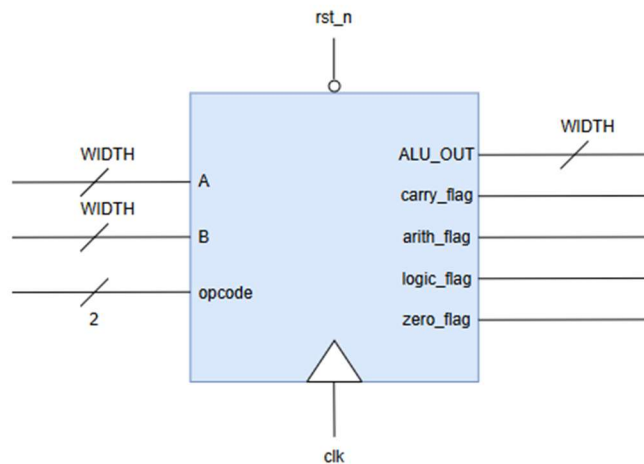


Figure 1: ALU port diagram

- 8-bit signed Arithmetic Logic Unit ($WIDTH = 8$).
- Reset signal **rst_n** is asynchronous, active low.
- Two **WIDTH**-bit signed operands, **A** and **B**.
- 2-bit **opcode** selects the operation: **ADD**, **SUB**, **AND**, **XOR**.
- **ALU_OUT** is a **WIDTH**-bit registered result, updated on the positive edge of **clk**.
- **carry_flag** is active high for **ADD/SUB** whenever the operation produces a carry-out / borrow beyond the MSB; registered.
- **arith_flag** is active high whenever **opcode** is **ADD** or **SUB**; registered.
- **logic_flag** is active high whenever **opcode** is **AND** or **XOR**; registered.
- **zero_flag** is active high whenever $ALU_OUT = 0$; combinational derived from the registered **ALU_OUT**.
- On reset ($rst_n = 0$), **ALU_OUT**, **carry_flag**, **arith_flag**, and **logic_flag** are all forced to 0.
- Unlike a priority-encoded control scheme, exactly one operation is selected per cycle directly by **opcode** — there is no signal-priority arbitration in this design.

Signals

1. **RST** (`rst_n`) -> Asynchronous, active low; all registered output signals (`ALU_OUT`, `carry_flag`, `arith_flag`, `logic_flag`) must be forced to zero immediately, independent of `clk`.
2. **A, B** -> `WIDTH`-bit signed operands; driven with valid constrained-random or corner values (see Sequences).
3. **opcode** -> 2-bit operation select, sampled at `posedge clk`:
 - **ADD** -> `ALU_OUT` <= A + B; `carry_flag` <= carry-out of the addition.
 - **SUB** -> `ALU_OUT` <= A - B; `carry_flag` <= borrow-out of the subtraction.
 - **AND** -> `ALU_OUT` <= A & B; `carry_flag` <= 0.
 - **XOR** -> `ALU_OUT` <= A ^ B; `carry_flag` <= 0.
4. **ALU_OUT** -> Checker (golden reference model):
 - If reset asserted -> check `ALU_OUT == 0`.
 - Else, at `posedge clk`, check `ALU_OUT` equals the result of the operation selected by `opcode` on the sampled A/B (per the golden model above).
5. **carry_flag** -> Checker:
 - If reset asserted -> check `carry_flag == 0`.
 - Checker: if `opcode` is **ADD** or **SUB** -> check `carry_flag` equals the reference carry/borrow bit.
 - Checker: if `opcode` is **AND** or **XOR** -> check `carry_flag == 0`.
6. **arith_flag** -> Checker:
 - If reset asserted -> check `arith_flag == 0`.
 - Checker: if `opcode` is **ADD** or **SUB** -> check `arith_flag == 1` on the following `posedge clk`.
 - Checker: otherwise -> check `arith_flag == 0`.
7. **logic_flag** -> Checker:
 - If reset asserted -> check `logic_flag == 0`.
 - Checker: if `opcode` is **AND** or **XOR** -> check `logic_flag == 1` on the following `posedge clk`.
 - Checker: otherwise -> check `logic_flag == 0`.
8. **zero_flag** -> Checker:
 - Checker: if `ALU_OUT == 0` -> check `zero_flag == 1`.
 - Checker: else -> check `zero_flag == 0`.
 - Sampled combinational, tracking the registered `ALU_OUT` — all the time, not pulsed.

Functional Coverage

We need to cover all output signals and their controlling inputs:

- **A** -> cover corner values: `A == 0` (ZERO), `A == max positive` (MAXPOS), `A == max negative` (MAXNEG), and default (all other values).
- **B** -> cover corner values: `B == 0` (ZERO), `B == max positive` (MAXPOS), `B == max negative` (MAXNEG), and default (all other values).
- **A** (logical ops only) -> cover walking-ones patterns on `A` {1,2,4,8,16,32,64,-128}, sampled only when `opcode` is **AND** or **XOR**.
- `opcode` -> cover **ADD**, **SUB** (arithmetic bin) and **AND**, **XOR** (logic bin).
- `opcode` -> cover transition sequence **ADD** => **SUB** => **AND** => **XOR**.
- Cross `opcode` x `A` -> every `opcode` exercised against every `A` corner bin.

- Cross `opcode` x `B` -> every opcode exercised against every `B` corner bin.
- `carry_flag` -> 1 iff an `ADD/SUB` produces a carry-out / borrow.
- `arith_flag` -> 1 iff `opcode` is `ADD` or `SUB`.
- `logic_flag` -> 1 iff `opcode` is `AND` or `XOR`.
- `zero_flag` -> 1 iff `ALU_OUT == 0`, covered across all four opcodes.
- Coverage is sampled only when `rst_n == 1` (samples during reset are excluded).

Sequences

Sequences & Constrained Randomization

9. Reset sequence -> `rst_n` driven low with weight 3/100 per transaction, deasserted with weight 97/100 (reset is rare, matching realistic operating conditions).
10. Arithmetic operand sequence (`opcode = ADD/SUB`) -> `A` and `B` independently weighted: MAXPOS, MAXNEG, and ZERO each at weight 90, remaining mid-range values collectively at weight 10 — biasing toward carry/overflow and zero-result corner cases.
11. Logical operand sequence (`opcode = AND/XOR`) -> `A` constrained to a walking-ones value {1,2,4,8,16,32,64,128} with weight 80, or a non-walking-ones value with weight 20; `B` constrained `!= A`.
12. Directed reset transaction -> the first transaction of every run forces `rst_n = 1`, `A = 0`, `B = 0`, `opcode = ADD` to establish a known post-reset baseline before randomization begins.

Test Scenarios

- **Scenario 1:** Full constrained-random regression -> 100,000 back-to-back transactions, `opcode` freely randomized across `ADD`, `SUB`, `AND`, `XOR` each cycle.
 - Goal: exercise every opcode against corner and default operand values, drive the full functional coverage model (including the `opcode` x `A/B` crosses) to closure, and stress the `carry_flag/zero_flag` checkers under sustained randomization.
- **Scenario 2:** Carry / borrow stress -> `ADD/SUB` transactions with `A`, `B` pinned at MAXPOS/MAXNEG/ZERO (weight-90 bins).
 - Goal: force repeated carry-out and borrow-out events on `carry_flag` and confirm `zero_flag` asserts correctly when an `ADD/SUB` result lands on zero (e.g. MAXPOS + MAXNEG-adjacent combinations).
- **Scenario 3:** Logical walking-ones sweep -> `AND/XOR` transactions with `A` driven through the walking-ones set and `B != A`.
 - Goal: cover every single-bit `A` pattern crossed with `AND/XOR`, and confirm `logic_flag` and `carry_flag == 0` hold throughout.
- **Scenario 4:** Opcode transition coverage -> sequence `ADD -> SUB -> AND -> XOR` repeated with random operands.
 - Goal: close the `opcode` transition bin and confirm `arith_flag/logic_flag` switch correctly one cycle after each opcode change.
- **Scenario 5:** Asynchronous reset scenario -> `rst_n` toggled low at random points during active randomization (weight 3/100 per transaction).
 - Goal: confirm `ALU_OUT`, `carry_flag`, `arith_flag`, and `logic_flag` clear immediately on reset assertion regardless of `clk`, and that operation resumes correctly once `rst_n` deasserts.